

OsmNav

Navigace autonomního robota do cíle nad daty OpenStreetMap

Popis architektury — Verze 2 (goal-rooted pole cost-to-goal)

C#/NET 9 · edge-based graf · D*/LPA* · TDD · 46/46 testů, 0 warningů

1. Účel

OsmNav je knihovna pro navigaci autonomního robota do cíle nad grafem postaveným z výřezu OpenStreetMap (formát **.osm XML**). Robot se pohybuje po mapě, průběžně určuje svou polohu a knihovna mu z každé pozice řekne, na které hraně se nachází a ke kterému uzlu má směřovat. Za jízdy se mohou měnit váhy cest a objevovat dopravní značky; systém na to reaguje bez počítání trasy od nuly.

Klíčové požadavky:

- Hledání nejrychlejší cesty grafem z OSM dat (výběr akceptovatelných typů cest, bariéry, *access*).
- Inkrementální přeplánování při změně vah hran za běhu.
- Omezení: jednosměrky, zákazy odbočení, přikázané směry.
- Runtime dopravní značky: omezení rychlosti, zákazy/přikazy odbočení, uzavírky.
- Více dopravních profilů (Auto / Pěší / Cyklo).
- Podpora více paralelních hypotéz o poloze (např. Monte Carlo lokalizace) nad jednou mapou.

2. Klíčová rozhodnutí

Edge-based graf (line graph)

Zákazy odbočení a přikázané směry nelze přirozeně zapsat do klasického grafu (uzly = křižovatky), protože v uzlu není známo, odkud vozidlo přijelo. Proto jsou **uzly grafu orientované hrany** původní mapy a **přechody mezi nimi jsou odbočení**. Jednosměrka = hrana jen v povoleném směru; zákaz odbočení = přechod se nevytvoří; přikázaný směr = ponechá se jen povolený přechod.

Goal-rooted pole cost-to-goal (bez startu)

Plánovač nepočítá jednu trasu start→cíl, ale **pole ceny do cíle** pro celý dosažitelný graf (inkrementální Dijkstra/LPA* počítaná **od cíle**). Robot z libovolné polohy jen „sestupuje po gradientu“. Pole je nezávislé na startu, takže ho může sdílet libovolný počet hypotéz o poloze. Počítá se **líně** — jen tak daleko, kam je zrovna potřeba.

Neměnná sdílená síť + dočasný cíl

Topologie a základní váhy jsou v **neměnné sdílené síti** (načtené jednou). Cíl se do grafu vloží jako **dočasný uzel**: nejbližší hrana se rozdělí na dvě regulérní poloviny (hrana do cíle má na konci cenu 0, hrana z cíle větší než 0). Přeplánování na nový cíl = dočasné uzly odstranit a vložit nové. Paralelní hypotézy do sítě jen čtou — žádné kolize, mapa v paměti jen jednou.

3. Komponenty

RoadNetwork	Neměnná edge-based síť (uzly = orientované hrany), base váhy, hledání reverzní hrany, sestavení přes Builder.
-------------	---

GoalField	Pole cost-to-goal (LPA* od cíle). Reálný split cílové hrany dočasným uzlem, líné EnsureSettled, CostToGoal, NextEdge, ClearGoal, runtime přepisy vah.
Router	Bezstavová extrakce trasy z bodu do cíle sestupem gradientu.
Navigator	Stavový sledovač: mapmatch polohy na hranu, volba směru dle ceny do cíle, hlášení (aktuální hrana, cílový uzel), detekce příjezdu.
SignApplier	Promítá runtime dopravní značky (rychlost, uzavírka, zákaz/příkaz odbočení) do pole.
OSM vrstva	Čtení .osm XML, dopravní profily (highway/oneway/barrier/access), stavba grafu včetně restrikcí odbočení.

4. Jak to funguje

Tok dat

.osm XML → čtečka (uzly/cesty/restrikce) → filtr profilem a stavba **RoadNetwork** (orientované hrany + přechody se zákazy odbočení) → **GoalField** (vložení cíle + pole cost-to-goal) → **Navigator/Router** čtou pole a vedou robota.

Cost-to-goal pole

- Pole se počítá od cíle; pro každou hranu udává cenu (čas) do cíle.
- **EnsureSettled(hrana)** dopočítá pole jen k dané hraně (líně, startem ohraničené). Usazené hodnoty jsou definitivní a sdílené.
- Změna váhy (dopravní značka, uzavírka) označí dotčené uzly a příští EnsureSettled opraví jen ovlivněnou část — ne celý graf.
- Robot v uzlu: další krok = soused minimalizující (cena přechodu + cena souseda do cíle).

Volba směru a příjezd

Na obousměrné hraně Navigator vybere orientaci s nižší celkovou cenou do cíle (zohlední i zbývající průjezd aktuální hranou), takže robot jede tou stranou, kterou je cíl blíže. Příjezd se hlásí, jakmile je robot dostatečně blízko cíle. Odchyłka od trasy nevyžaduje explicitní přeplánování — jiná poloha prostě přečte pole jinde.

5. Dopravní pravidla a runtime značky

Robot za běhu najde	Operace	Efekt
Omezení rychlosti	SetTraversalCost(hrana, délka/rychlost)	zdraží průjezd danou ulicí
Zákaz odbočení	SetTurnCost(z, do, ∞)	odbočení zmizí z řešení
Přikázaný směr	SetTurnCost = ∞ všem kromě povoleného	vynutí jediný směr
Uzavírka / překážka	SetTraversalCost(hrana, ∞)	ulice se obejde

Značky jsou brány jako globální (jeden model světa), takže jedno sdílené pole obslouží všechny hypotézy o poloze. Změny se do pole promítají inkrementálně.

6. Paralelní hypotézy (MCL)

Protože pole cost-to-goal nezávisí na startu, může ho **sdílet libovolný počet hypotéz** o poloze robota (částice Monte Carlo lokalizace). Neměnná síť i jedno pole jsou v paměti jen jednou; každá hypotéza jen čte svou lokální hodnotu a gradient. Když hypotéza potřebuje hodnotu z dosud nespočítané oblasti, líné

dopočítání pole rozšíří hranici — a výsledek rovnou využijí i ostatní.

7. Stav a kvalita

- Vývoj testovaně (TDD): každá jednotka nejdřív test, pak implementace.
- **46/46 testů zelených, 0 varování** (build s -warnaserror).
- Korektnost pole ověřena **nezávislým orákulem** (ručně předrozdělená síť + prostá Dijkstra) pro jednosměrné i obousměrné cesty.
- Ověřeno sdílení jednoho pole více hypotézami a inkrementální oprava při změně vah.

8. Rozsah a omezení

- Vstup jen .osm XML (menší výřezy); malá mapa v paměti.
- Restrikce odbočení typu via-node (vzácné via-way se přeskočí).
- Heuristické zaostření hledání (koridor místo „koule“) je připraveno jako budoucí volitelný režim.
- Jednovláknové počítání pole; plná paralelizace čtení je možným dalším krokem.